Behavior Cloning With Fuzzy Logic for Accelerating Early-Stage DRL in Edge Offloading

Jiujie Zhang, *Graduate Student Member, IEEE*, Yangfei Lin, *Member, IEEE*, Zhaoyang Du, *Member, IEEE*, Lei Zhong, *Member, IEEE*, Wugedele Bao, Ha Si, Soufiene Djahel, *Senior Member, IEEE*, and Celimuge Wu, *Senior Member, IEEE*

***Abstract*—Deep Reinforcement Learning (DRL) has proven effective in addressing the complexities of task offloading in heterogeneous and dynamic edge computing environments. However, the iterative training process of DRL, particularly in its early stages, is computationally intensive and time-consuming, posing significant challenges for delay-sensitive applications. Furthermore, DRL agents are typically initialized with random weights, as is common in deep learning, resulting in models that lack meaningful behavioral priors and further complicate the learning process. To tackle these issues, this paper leverages fuzzy logic to improve the early-stage performance of DRL agents in task offloading through behavior cloning. By exploiting the high-level reasoning capabilities of fuzzy logic, behavior cloning equips DRL agents with heuristic knowledge, thereby reducing exploration costs and accelerating training in the early learning stages. To facilitate the seamless integration of fuzzy logic–based knowledge into DRL agents, five distinct behavior cloning strategies are introduced: only cloning, norm cloning, fading cloning, runtime cloning, and hybrid cloning. Extensive simulations in multi-access edge computing environments with multiple edge servers and user devices demonstrate that integrating fuzzy logic with DDQN-based reinforcement learning via behavior cloning significantly improves the Average Task Completion Rate (ATCR) and reduces the Average Task Processing Time cost (ATPT) of all user devices over learning episodes during early training stages, while concurrently enhancing the stability of learning progress compared to DRL baseline approach.**

***Index Terms*—Behavior cloning, deep reinforcement learning, fuzzy logic, task offloading, edge computing.**

I. Introduction

A. Background

DATA-DRIVEN digitalization and intelligent technologies have transformed society, leading to improvements in productivity and daily life. Although cloud computing and data centers address diverse computational needs, their geographical distance from terminal devices results in increased latency and energy consumption, thereby degrading the quality of service (QoS) and user experience (QoE). These limitations hinder their ability to meet the demands of computing-first networking and modern consumer electronics. Edge computing, including multi-access edge computing (MEC) [1], [2], has emerged as a promising solution. Positioned between cloud centers and terminals, edge nodes leverage low-latency radio access networks (RAN) to enhance computational efficiency and communication performance. Consequently, resource-constrained end devices can rely on edge servers for task processing. In MEC, multiple edge servers are deployed to satisfy user requirements [3], which raises the challenge of deciding whether to process tasks locally or offload them to an optimal server, thereby introducing a problem known as task offloading [4], [5]. It is typically formulated as a multi-objective optimization problem that needs to balance completion rate, time, energy, and other requirements [6].

B. Issues of Concern

In recent years, a variety of solutions have been proposed to address task offloading in edge computing, including traditional methods, evolutionary algorithms [6], fuzzy logic methods [7], and machine learning approaches [8], among others. Deep reinforcement learning refines strategies iteratively through interaction with the environment, aiming to achieve an optimal policy. It has emerged as a leading paradigm for task offloading in edge computing in recent literature, owing to its ability to optimize decision-making processes [9].

Despite its advantages, DRL inherits a fundamental limitation from deep learning: the dependence on random parameter initialization, which leads to unclear initial behavioral tendencies and exacerbates the cold-start problem. Deep learning mitigates this issue through large-scale sampling and rapid gradient descent. In contrast, needing for sequential and step-by-step interactions with the environment, DRL agents amplifies the problem. This frequently overlooked aspect

Received 13 March 2025; revised 3 May 2025; accepted 19 June 2025. Date of publication 27 June 2025; date of current version 7 November 2025. This work was supported in part by the Natural Science Foundation of Inner Mongolia Autonomous Region, China, under Grant 2025QN06036 and Grant 2025MS06044; in part by the Japan Science and Technology Agency (JST) through the ASPIRE Program under Grant JPMJAP2325; and in part by the Toyota Motor Corporation, Japan. *(Corresponding author: Celimuge Wu.)*

Jiujie Zhang, Yangfei Lin, and Zhaoyang Du are with the Department of Computer and Network Engineering, The University of Electro-Communications, Tokyo 182-8585, Japan (e-mail: zhangjiujie@uec.ac.jp; linyangfei@uec.ac.jp; duzhaoyang@uec.ac.jp).

Lei Zhong is with the Information and Communication Planning Division, Toyota Motor Corporation, Tokyo 100-0004, Japan (e-mail: lei_zhong@mail.toyota.co.jp).

Wugedele Bao and Ha Si are with the Trustworthy Big Data and AI Academician Workstation, Hohhot Minzu College, Hohhot 010051, China (e-mail: wugedele.imnc@gmail.com; hasi76@126.com).

Soufiene Djahel is with the Centre for Future Transport and Cities, Coventry University, CV1 5FB Coventry, U.K. (e-mail: ae3095@coventry.ac.uk).

Celimuge Wu is with the Meta-Networking Research Center and the Department of Computer and Network Engineering, The University of Electro-Communications, Tokyo 182-8585, Japan (e-mail: celimuge@uec.ac.jp).

Digital Object Identifier 10.1109/TCE.2025.3583767

1558-4127 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

in research not only prolongs training times and increases computational costs, but also significantly limits the early-stage performance of DRL-based task offloading algorithms in edge computing.

However, providing DRL agents with initial parameters that exhibit clear behavioral semantics remains challenging due to the lack of bidirectional interpretability mapping between parameters and behaviors. Furthermore, this challenge is amplified by the scarcity of labeled data for task offloading, due to the diversity of system architectures and varying requirements.

C. Research Motivation

Generally, DRL algorithms perform well in later stages but suffer from excessive exploration during early stages due to the cold-start issue from random initialization. In contrast, fuzzy logic offers human-like decision-making capabilities and robust initial performance. Integrating the strengths of fuzzy logic into DRL algorithms could not only enhance early-stage performance but also improve interpretability. Fortunately, imitation learning [10], especially behavior cloning (BC) [11], [12], provides greater promise for achieving this goal in the context of edge offloading. A feasible solution to the cold-start issue is to design a fuzzy logic module that accumulates experience and transfers it to DRL agents via behavior cloning. To the best of our knowledge, no prior research has systematically addressed this challenge in typical edge task offloading scenarios.

Although fuzzy logic is not the only option, we incorporated it into the behavior cloning framework based on several key considerations. First, it is lightweight and computationally efficient, requiring neither iterative optimization nor large-scale data, which suits resource-constrained edge environments. Second, its human-like reasoning enhances interpretability, thereby supporting more effective experience cloning. Third, its rule-based structure, combined with membership function-based adaptability, provides strong initial decision-making and responsiveness to dynamic environments, thereby mitigating cold-start issues. Lastly, fuzzy logic is easy to implement and adjust, making it practical for real-world, latency-sensitive edge systems.

D. Research Objective

Building on the background and motivation above, this article focus on refining a fuzzy logic-based offloading module with acceptable performance for delay-sensitive edge offloading, and utilizing experience data generated by fuzzy logic to enhance the early-stage performance of DRL agents via behavior cloning. The research encompasses several key aspects: modeling delay-sensitive edge computing scenario; developing fuzzy logic component, including variable design, membership functions, and inference rules for multi-user task offloading; implementing DRL baseline algorithm and behavior cloning workflow; and evaluating the proposed approach through comprehensive analysis focus on improving the early performance of DRL algorithm.

E. Key Contributions and Advances Over Prior Work

This paper enhances the early task offloading performance of DRL agents through fuzzy logic-guided behavior cloning. Building on our preliminary work [13], we propose a refined framework that integrates fuzzy reasoning with DRL, offering novel contributions in both methodology and experimentation. The key contributions and improvements over prior work are as follows:

- A multi-user, multi-server, latency-sensitive task offloading model that supports distributed decision-making in dynamic edge environments.
- An enhanced fuzzy logic offloading module featuring optimized rule inference and hyperparameter tuning for smoother and more robust decision support.
- Five customized behavior cloning strategies for transfering fuzzy logic expertise into DRL agents, forming a structured workflow to addresses the cold-start and problem.
- A comprehensive experimental evaluation of the impact of fuzzy-guided behavior cloning on the early performance of DRL under varying user densities, task sizes, arrival rates, and BC configurations.

The rest of this paper is organized as follows. Section II reviews related works, followed by Section III, which presents the MEC task offloading model and formulates the joint-optimization problem targeting high task completion rates and low processing times. Subsequently, Section IV introduces the baseline methods incorporating fuzzy logic and DRL, along with the behavior cloning framework. Building upon these foundations, Section V presents and analyzes the simulation results. Finally, Section VI concludes the study while outlining promising future research directions.

II. Related Works

A. Edge Computing Task Offloading

Task offloading in edge computing enhances system performance by transferring tasks from resource-constrained devices to edge or cloud servers. It can be categorized by decision-making entities (centralized vs. distributed) [14], target destinations (edge, cloud, or peer devices) [15], and granularity (full vs. partial) [16]. Offloading may occur via single-hop (direct transfer) or multi-hop (through intermediate nodes) [17], and strategies range from rule-based and optimization-driven to machine learning-enabled. Offloading directions may also be unidirectional or bidirectional [18]. Various frameworks, including DRL-based [19] and fuzzy logic-based approaches [20], have been proposed to improve offloading efficiency.

B. DRL-Based Task Offloading

Existing studies highlight the widespread use of DRL in task offloading optimization [21]. Wu et al. [22] and Dai et al. [23] combined federated learning with DRL to enhance resource utilization and reduce system costs in distributed frameworks. Pang et al. [24] introduced a channel-aware adaptive offloading method to minimize overhead, while

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

Liang et al. [25] applied the Lyapunov optimization with DRL to balance cost and stability in vehicular edge networks. Rahmati et al. [26] developed a QoE-driven DRL method to improve task completion and energy efficiency. Compared to conventional methods, DRL-based approaches exhibit superior adaptability by dynamically optimizing offloading through massive interactive steps. Further details are provided in in [27], [28].

C. Fuzzy Logic-Based Task Offloading

Fuzzy logic has attracted attention in task offloading due to its capacity to handle uncertainty and enhance decision-making. Mahapatra et al. [29] proposes a fuzzy-based scheduling mechanism that improved task allocation efficiency by considering latency, energy, and load balancing. Abdulazeez and Askar [30] employed fuzzy logic to optimize task offloading, achieving better latency, power consumption, and network throughput. In Internet of Things (IoT) systems, Nguyen et al. [31] developed a fuzzy-based orchestrator to split tasks between cloud and edge, reducing latency and task failures while enhancing service time. Lu et al. [32] addressed energy minimization in cognitive radio-based MEC systems using non-orthogonal multiple access and DRL algorithm with convex optimization to save energy. Rai et al. [33] applied fuzzy logic in vehicular fog computing to optimize task allocation for community value and resource sharing. Additional works are available in [34], [35].

D. Hybrid Methods for Task Offloading

Hybrid approaches have been extensively explored to address task offloading challenges. Behera et al. [36] proposed a fuzzy-based method incorporating classification and regression trees. The integration of reinforcement learning with fuzzy logic has also been studied; for example, Son et al. [37] introduced a fuzzy Q-learning scheme to enhance QoE in vehicular fog computing. Wang et al. [38] applied imitation learning to stabilize DRL agents, improving response times in dynamic vehicular edge environments. Xu et al. [39] developed the F-TORA scheme by combining Takagi-Sugeno fuzzy and neural networks with game theory to reduce latency. Sui et al. [40] designed a multi-strategy fusion algorithm for cloud-fog scenarios, accelerating convergence and reducing execution time and costs. UAV-assisted offloading has also attracted interest in emerging applications [41]. As underlying theories and technologies advance, more hybrid methods are expected to emerge for edge task offloading.

E. Summary and Research Opportunities

Recent studies in edge computing recognize DRL as a powerful solution for task offloading, with growing interest in combining it with fuzzy logic to improve efficiency. However, most existing works overlook the cold-start problem during DRL policy learning. Fuzzy logic offers strong early-stage performance through human-like reasoning, whereas DRL relies on extensive explorations, leading to inefficiency at the beginning. The integration of fuzzy logic into the early phase via behavior cloning not only improves the initial performance of DRL agents but also provides valuable insights for optimizing DRL-based federated task offloading strategies.

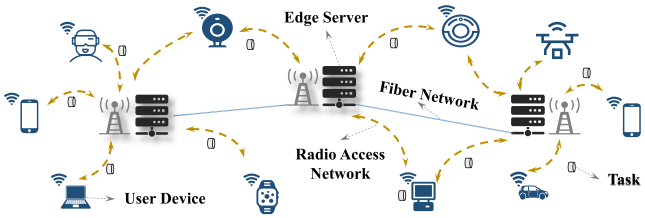


Fig. 1. System model of MEC task offloading environment.

III. System Model and Problem Formulation

A. System Model

This study aims to enhance the early performance of DRL agents for task offloading in latency-sensitive edge computing scenarios. We model a distributed environment where end devices independently observe and make decisions, enabling personalized, privacy-preserving operations without mutual interference. Building on prior work [13], the system model is concisely structured into four components: infrastructure, task scheduling, transmission, and processing.

1) Infrastructure Component: The edge task offloading scenario involves multiple edge servers (ESs) and multiple user devices (UDs) communicating over low-latency RAN. Fig. 1 illustrates an abstraction of this setup. Edge servers are denoted by $\mathcal{S} = \{es_1, es_2, \ldots, es_S\}$, and user devices by $\mathcal{D} = \{ud_1, ud_2, \ldots, ud_D\}$, with cardinalities S and D, respectively. Time is divided into episodes, each consisting of discrete time slots $\mathcal{T} = \{1, 2, \ldots, T\}$, of equal duration δ seconds. Computing resources are distributed between UDs and ESs. Each $ud_i \in \mathcal{D}$ has a CPU frequency f_i^{d}, while each $es_j \in \mathcal{S}$ operates at f_j^{s}. A user device maintains two queues: $q_{i,cp}^{\mathrm{d}}$ for local execution and $q_{i,tr}^{\mathrm{d}}$ for offloading via RAN. Each ES allocates resources across D queues, one per UD; For instance, $q_{i,j}^{\mathrm{s}}$ in es_j handles tasks from ud_i. Resources are evenly shared among active queues within an ES.

2) Task Scheduling Component: Tasks are indivisible and mutually independent. In time slot t, a task $k_i(t)$ arrives at ud_i with size $z_i(t)$ (in bits), selected from a discrete set $\mathcal{Z} = \{0\} \cup \{z_1, z_2, \ldots, z_Z\}$. $z_i(t) = 0$ indicates no task arrival at slot t at ud_i. Each task requires $\rho_i(t)$ CPU cycles per unit and must be completed with in its deadline $\tau_i(t)$, i.e., by time $t + \tau_i(t) - 1$. Tasks missing the deadlines are dropped. Upon arrival, the system determines the processing location. If processed locally, task is placed into $q_{i,cp}^{\mathrm{d}}$; otherwise, it enters $q_{i,tr}^{\mathrm{d}}$ and is transmitted via RAN to the queue $q_{i,j}^{\mathrm{s}}$ in the selected es_j, as shown in Fig. 2. To represent the state of ESs, an integer $b_j(t)$ with approximately D binary digits encodes the activity of all queues in es_j at time slot t, where each bit denotes wether a queue is active. The historical activity across all ESs over the past T_{past} slots is recorded as $\boldsymbol{b}(t) = \{b_{j\in\mathcal{S},t'\in[t-T_{\mathrm{past}},t-1]}(t')\}$. This encoding reduces communication overhead and enhances system efficiency.

3) Task Transmission Component: All UDs use orthogonal channels to offload tasks to ESs, which are interconnected via

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

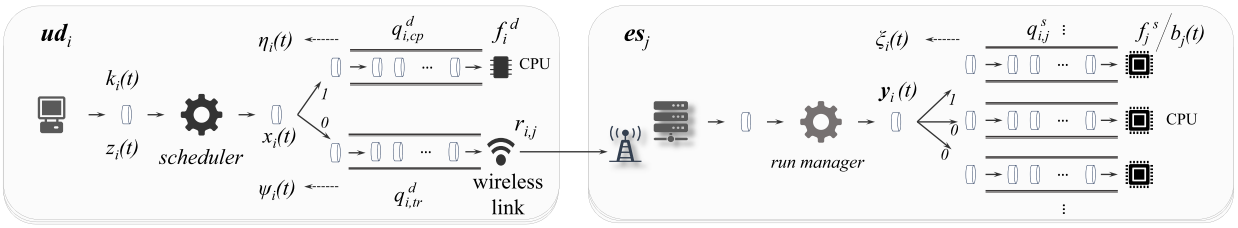


Fig. 2. An view of task offloading in edge computing system with the ud_i and es_j.

fiber networks to ensure robust and efficient communication. When task $k_i(t)$ is offloaded to es_j, the waiting time in queue $q^{\mathrm{d}}_{i,tr}$ is denoted as $\psi^{\mathrm{wt}}_i(t)$, and the transmission time as $\psi^{\mathrm{sd}}_i(t)$. Both are continuously estimated and updated in real-time. Let $\ell^{\mathrm{d}}_{i,tr}(t)$ be the length of $q^{\mathrm{d}}_{i,tr}$ at slot t. The estimated waiting time is $\psi^{\mathrm{wt}}_i(t) = \ell^{\mathrm{d}}_{i,tr}(t)/r_{i,j}$, and transmission time is $\psi^{\mathrm{sd}}_i(t) = z_i(t)/r_{i,j}$, where $r_{i,j}$ denotes the transmission rate between ud_i and es_j. Considering the effects of path loss and small-scale fading, $r_{i,j}$ is calculated as:

$$r_{i,j} = W log_2\big(1 + \frac{h_{i,j}P^{tr}_i}{\sigma^2}\big). \tag{1}$$

Where W is the channel bandwidth, $h_{i,j}$ is the channel gain, P^{tr}_i is the transmit power, and σ^2 is noise power. Following [42], [43], we assume a fixed transmission rate $r_{i,j}$. Thus, the total estimated transmission time is calculated as:

$$\psi^{\mathrm{tr}}_i(t) = \frac{z_i(t) + \ell^{\mathrm{d}}_{i,tr}(t)}{r_{i,j}}. \tag{2}$$

Consistent with [26], [43], The downlink time is considered negligible due to smaller data size and faster speed, so optimization focuses on uplink offloading and task execution, aligning with practical MEC scenarios.

4) Task Processing Component: On user device side, let $\eta^{\mathrm{wt}}_i(t)$, $\eta^{\mathrm{exe}}_i(t)$, and $\eta^{\mathrm{fn}}_i(t)$ denote the waiting, execution, and completion times for task $k_i(t)$, respectively. Let $\ell^{\mathrm{d}}_{i,cp}(t)$ be the length of $q^{\mathrm{d}}_{i,cp}$ at the beginning of time slot t. The estimated completion time for local processing is calculated as follows:

$$\eta^{\mathrm{fn}'}_i(t) = \frac{z_i(t) + \ell^{\mathrm{d}}_{i,cp}(t)}{f^{\mathrm{d}}_i\,\delta/\rho_i(t)}. \tag{3}$$

On the edge server side, let $\xi^{\mathrm{wt}}_{i,j}(t)$, $\xi^{\mathrm{exe}}_{i,j}(t)$, and $\xi^{\mathrm{fn}}_{i,j}(t)$ denote the respective processing stages when task $k_i(t)$ is executed by es_j. Let $aq^{\mathrm{s}}_j(t)$ be the number of active queues in es_j after time slot $t-1$, and $\ell^{\mathrm{s}}_{i,j}(t)$ be the length of queue $q^{\mathrm{s}}_{i,j}$ at slot t. Real-time values of $aq^{\mathrm{s}}_j(t)$ and $\boldsymbol{\ell}^{\mathrm{s}}_i(t) = \{\ell^{\mathrm{s}}_{i,j\in\mathcal{S}}(t)\}$ enable estimation of the edge-side completion time as:

$$\xi^{\mathrm{fn}'}_{i,j}(t) = \frac{z_i(t) + \ell^{\mathrm{s}}_{i,j}(t)}{f^{\mathrm{s}}_j\,\delta\,/\left(\rho_i(t)\,aq^{\mathrm{s}}_j(t)\right)}. \tag{4}$$

Due to the dynamic nature of ES queues, actual processing time must also be recorded in real time. This offloading system shares key assumptions with related works [26], [42], [43], such as task queuing and indivisibility, making it consistent with typical MEC environments.

The modeled environment features a multi-user, multi-edge server environment with task queueing and timeout-based drop on both the local and edge sides. Each edge server maintains per-user queues and dynamically allocates resources to active ones. The problem formulation and solutions are then built on these characteristics. Unlike single-server or shared-queue models, the proposed strategy leverages multiple servers and dedicated queues, offering greater generalizability and better alignment with real-world edge computing scenarios.

B. Problem Formulation

The goal of the offloading policy is to optimize long-term performance by minimizing total processing time and ensuring a high average task completion rate.

1) Problem Abstraction: To minimize the long-term expected average task processing time while maximizing the average task completion rate, we formulate the task offloading problem as a Markov Decision Process (MDP). This abstraction is consistent with related works [26], [42], [43], where DRL is effectively employed to capture the dynamic nature of task offloading and processing. In this work, we adopt a DRL baseline method for edge computing task scheduling, aiming to jointly optimize task processing time and the completion rate over the long term. Consequently, we define the key components of reinforcement learning, including observability, state and action, reward and cost, and the optimization objective.

2) Observability: Each device observes the environment and makes offloading decisions via a DRL agent. The observation scope includes the following.

- Task attributes: size, computational density, and timeout constraints.
- Device status: local computing capability and lengths of both computation and transmission queues.
- Server status: computational capacity, queue activity, and the queue lengths and backlogs of each edge server.

User devices are assumed to have prior knowledge of the characteristics of the resources and networks. Edge servers periodically broadcast their queue states as compact numerical values over low-latency wireless networks, ensuring minimal communication overhead. This design is consistent with typical practices in the literature [26], [43].

3) State and Action: In the DRL algorithm build upon the MDP framework, each user device $ud_i \in \boldsymbol{D}$ collects relevant environmental information and organizes it into a sate vector. The state of ud_i at slot t is defined as:

$$\boldsymbol{s}_i(t) = \big(z_i(t),\ \eta^{\mathrm{wt}}_i(t),\ \psi^{\mathrm{wt}}_i(t),\ \boldsymbol{\ell}^{\mathrm{s}}_i(t-1),\ \boldsymbol{b}(t)\big). \tag{5}$$

The action of an agent, i.e., the offloading decision, is described by two components. Let $x_i(t) \in \{0, 1\}$ indicate

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

whether task $k_i(t)$ processed locally ($x_i(t) = 1$), or offloaded to an edge server ($x_i(t) = 0$). If $x_i(t) = 1$, the task is placed in the local queue $q^{\mathrm{d}}_{i,cp}$; otherwise, it is routed to an edge server. For tasks to be offloaded, the decision of selecting a specific edge server es_j is represented by $y_{i,j}(t) \in \{0, 1\}$, where the vector $\boldsymbol{y}_i(t) = \{y_{i,j\in\mathcal{S}}(t)\}$ encodes the selection status across all edge servers. The complete action for ud_i at time slot t is then formulated as:

$$\boldsymbol{a}_i(t) = \big(x_i(t),\, \boldsymbol{y}_i(t)\big). \tag{6}$$

4) Reward and Cost: The reward $w_i(t)$ obtained after executing $\boldsymbol{a}_i(t)$ under state $\boldsymbol{s}_i(t)$ is defined as follows:

$$w_i(t) = min\left(x_i(t)\,\eta^{\mathrm{fn}}_i(t) + \sum_{j=1}^{S} y_{i,j}(t)\,\xi^{\mathrm{fn}}_{i,j}(t),\; t + \tau_i(t) - 1\right). \tag{7}$$

This reward formulation captures the task completion time resulting from both local execution and offloading decisions, constrained by the task deadline. The task completion status $\beta_i(t) \in \{0, 1\}$ indicates whether task $k_i(t)$ is successfully completed ($\beta_i(t) = 1$) or dropped due to exceeding the deadline ($\beta_i(t) = 0$). To penalize task failures, a fixed penalty P is imposed for dropped tasks. Accordingly, the cost function is defined as:

$$c_i(t) = \beta_i(t)\, w_i(t) + (1 - \beta_i(t))\, P. \tag{8}$$

5) Optimization Problem: The objective of each agent in the offloading system is to learn an optimal policy $\boldsymbol{\pi}^*$ that minimizes the long-term expected cost. The policy is defined as a mapping from the state space to the action space, expressed as $\boldsymbol{\pi} : \boldsymbol{Sta} \rightarrow \boldsymbol{Act}$. At the start of slot t, the agent residing on ud_i selects the action $\boldsymbol{a}_i(t) \in \boldsymbol{Act}$ based on its observed state $\boldsymbol{s}_i(t) \in \boldsymbol{Sta}$. Let $\gamma \in (0, 1]$ denote the discount factor, which accounts for the diminishing importance of future states. The long-term objective is to minimize the expected discounted cumulative cost, formulated as:

$$\boldsymbol{\pi}^*_i = \mathrm{argmin}_{\boldsymbol{\pi}_i} \mathbb{E}\left[\sum_{t\in\mathcal{T}} \gamma^{t-1}\, c_i(t) \big| \boldsymbol{\pi}_i\right]. \tag{9}$$

In the proposed task offloading system, each agent independently observes the environment and makes autonomous offloading decisions. The overarching goal is to minimize the expected cost incurred by each individual agent over time.

IV. Offloading Methods and Behavior Cloning

This section introduces two baseline methods: a fuzzy logic-based offloading module and a DRL-based approach. These are followed by the presentation of behavior cloning strategies and the workflow.

A. Fuzzy Logic-Based Offloading Module

To obtain high-quality task offloading experiences before executing behavior cloning, we employ a fuzzy logic-based module that is simple, interpretable, and technically feasible. This module aims to perform effective task offloading in the edge computing environment described in Section III-A, with the goal of achieving high average task completion rates and low average processing times.

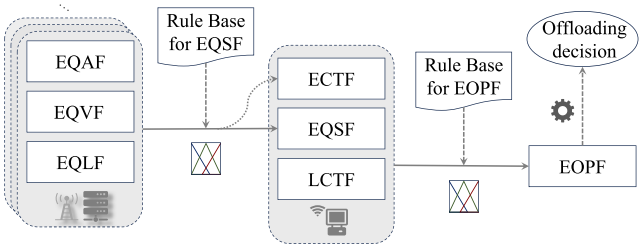


Fig. 3. Workflow of fuzzy logic-based task offloading module.

1) Design Idea: In multi-edge server scenarios, the fuzzy logic-based offloading adopts a two-stage process, as illustrated in Fig. 3. In the first stage, user device evaluates the quality of service of each server by computing the Edge Queue Service Factor (EQSF), which integrates the Edge Queue Activity Factor (EQAF), the Edge Queue Volume Factor (EQVF), and the Edge Queue Length Factor (EQLF). The server with the highest EQSF is selected as the candidate for offloading. In the second stage, user device combines the Local Completion Time Factor (LCTF), Edge Completion Time Factor (ECTF), and EQSF to compute the Edge Offloading Priority Factor (EOPF). The final decision is then made via defuzzification of the EOPF. These metrics, as originally introduced in [13], provide the foundation for the offloading strategy.

2) Fuzzy Variables and Rule Bases: Building on the above concept, the subsequent paragraphs introduce the fuzzy variables and describe their corresponding membership functions and inference rules.

EQAF (Edge Queue Activity Factor): This fuzzy variable, with levels *low*, *medium*, and *high*, indicates the queue activity of a edge server, measured by the ratio of active queues to the number of UDs. It reflects short-term activity, but does not account for long-term workload which depends on task backlogs. The EQAF of es_j at time slot t is computed as:

$$EQAF_j(t) = \frac{aq^s_j(t)}{D}. \tag{10}$$

EQVF (Edge Queue Volume Factor): While EQAF reflects short-term conditions, EQVF captures potential future workload based on task backlog. Categorized into *low*, *medium*, and *high*, EQVF is the normalized rank of a server's backlog among all servers. Given fixed computing power, larger backlogs imply increased future load, while smaller ones suggest idleness. Let $bl^{\mathrm{s}}_j(t)$ be the backlog at es_j in time slot t, its EQVF is computed as:

$$EQVF_j(t) = \frac{bl^{\mathrm{s}}_j(t)}{max\Big(\{bl^{\mathrm{s}}_{j\in\mathcal{S}}(t)\}\Big)}. \tag{11}$$

EQLF (Edge Queue Length Factor): This variable evaluates the relative queue length on edge servers from the perspective of user device. While shorter queues might seem advantageous, ignoring the server's overall workload can

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

TABLE I
RULE BASE FOR REASONING EQSF

Rule	EQAF	EQVF	EQLF	EQSF
Rule1	Low	Small	Short	Good
Rule2	Low	Small	Medium	Good
Rule3	Low	Small	Long	Good
Rule4	Medium	Small	Short	Good
Rule5	Medium	Small	Medium	Good
Rule6	High	Small	Medium	Good
Rule7	High	Small	Short	Good
Rule8	High	Small	Long	Good
Rule9	Medium	Small	Long	Good
Rule10	Medium	Medium	Medium	Medium
Rule11	Medium	Medium	Short	Medium
Rule12	Medium	Medium	Long	Medium
Rule13	Low	Large	Medium	Medium
Rule14	Low	Large	Long	Medium
Rule15	Low	Large	short	Medium
Rule16	Low	Medium	Medium	Medium
Rule17	Low	Medium	short	Medium
Rule18	Medium	Medium	Long	Medium
Rule19	High	Large	Long	Bad
Rule20	High	Large	Medium	Bad
Rule21	High	Large	Short	Bad
Rule22	Medium	Large	Long	Bad
Rule23	Medium	Large	Medium	Bad
Rule24	High	Medium	Long	Bad
Rule25	High	Medium	Medium	Bad
Rule26	Medium	Large	Short	Bad
Rule27	High	Medium	Short	Bad

TABLE II
RULE BASE FOR REASONING EOPF

Rule	LCTF	ECTF	EQSF	EOPF
Rule1	High	Low	Good	Outstanding
Rule2	High	Low	Medium	Outstanding
Rule3	Medium	Low	Good	Outstanding
Rule4	High	Medium	Good	Superior
Rule5	High	Medium	Medium	Superior
Rule6	High	Low	Bad	Superior
Rule7	Medium	Low	Medium	Superior
Rule8	High	High	Good	Comparable
Rule9	High	Medium	Bad	Comparable
Rule10	Medium	Medium	Good	Comparable
Rule11	Low	Low	Medium	Comparable
Rule12	Medium	Low	Bad	Comparable
Rule13	Medium	Medium	Medium	Comparable
Rule14	High	High	Medium	Average
Rule15	Medium	High	Good	Average
Rule16	Low	Medium	Good	Average
Rule17	Medium	High	Medium	Average
Rule18	Medium	Medium	Bad	Average
Rule19	Low	Low	Good	Average
Rule20	High	High	Bad	Subpar
Rule21	Medium	High	Bad	Subpar
Rule22	Low	High	Good	Subpar
Rule23	Low	Medium	Medium	Subpar
Rule24	Low	Low	Bad	Subpar
Rule25	Low	High	Bad	Inferior
Rule26	Low	High	Medium	Inferior
Rule27	Low	Medium	Bad	Inferior

lead to suboptimal offloading decisions. Categorized as *short*, *medium*, or *long*, EQLF is normalized, where smaller values indicates shorter queues and better suitability. The EQLF of an edge server for ud_i is computed based on its rank of $\ell^{\mathrm{s}}_{i,j}(t)$ among $\boldsymbol{\ell}^{\mathrm{s}}_i(t)$, and is calculated as follows:

$$EQLF_{i,j}(t) = \frac{\ell^{\mathrm{s}}_{i,j}(t)}{max\Big(\{\ell^{\mathrm{s}}_{i,j\in\mathcal{S}}(t)\}\Big)}. \tag{12}$$

EQSF (Edge Queue Service Factor): Each user device uses EQSF to select the best server, categorizing EQSF into *bad*, *medium*, and *good*, with higher values indicating better service quality. Inference rules for EQSF are provided in Table I. For instance, an edge server with low activity, small backlog, and short device queue provides higher service quality. After selecting the top EQSF server, the device compares local and edge processing options by estimating waiting, transfer, and computation times based on queue lengths and capacities. Finally, the offloading decision is made by integrating EQSF with local LCTF and edge ECTF, resulting in the final EOPF value.

LCTF (Local Completion Time Factor): The LCTF provides fuzzy timeout risks for local task processing, categorized as *low*, *medium*, and *high*. Given $z_i(t)$ and the queue length $\ell^{\mathrm{s}}_{i,j'}(t)$ on the selected server $es_{j'}$, ECTF is approximated using $f^{\mathrm{s}}_{j'}$ and $aq^{\mathrm{s}}_{j'}(t)$, as follows:

$$LCTF_i(t) = min\left(\frac{z_i(t) + \ell^{\mathrm{d}}_{i,cp}(t)}{\tau_i(t) f^{\mathrm{d}}_i\, \delta/\rho_i(t)},\ 1\right). \tag{13}$$

ECTF (Edge Completion Time Factor): The ECTF estimates fuzzy timeout on an edge server. Similar to LCTF, it is normalized and categorized into *low*, *medium*, and *high* levels, representing varying degrees of timeout risk. Specifically, given $z_i(t)$ and $\ell^{\mathrm{s}}_{i,j'}(t)$ of the selected server $es_{j'}$, ECTF can be approximately estimated using $f^{\mathrm{s}}_{j'}$ and $aq^{\mathrm{s}}_{j'}(t)$, as follows:

$$ECTF_i(t) = min\left(\frac{z_i(t) + \ell^{\mathrm{s}}_{i,j'}(t)}{\tau_i(t) f^{\mathrm{s}}_{j'}\, \delta/\Big(\rho_i(t)\, aq^{\mathrm{s}}_{j'}(t)\Big)},\ 1\right). \tag{14}$$

EOPF (Edge Offloading Priority Factor): The EOPF combines LCTF, ECTF, and EQSF to assess offloading preferences, using fuzzy levels such as *outstanding*, *superior*, *comparable*, *average*, *subpar*, and *inferior*, with higher levels favoring offloading. Defuzzification via the center of gravity (COG) method [44] produces a final decision value between 0 and 1. Table II presents the reasoning rules for EOPF, and the threshold (EOPF-TH) for offloading decisions is discussed in the experimental section.

3) Membership Functions: We adopt simple, intuitive membership functions for the fuzzy variables and levels, as shown in Fig. 4. Though not necessarily optimal, they perform effectively in practice. Future work will explore tuning for improved adaptability. In behavior cloning, this fuzzy module enhances reinforcement learning agents by providing strong initial offloading capabilities. Fig. 5 illustrates the module's continuous evaluation, where output smoothly varies with ECTF and EQSF under fixed LCTF (0.5), demonstrating its flexibility across scenarios. As one of the baselines, the fuzzy logic module offers an interpretable framework for handling uncertainty. Built upon prior work, it is integrated into our

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

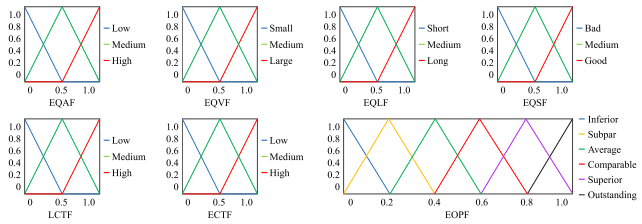


Fig. 4. Fuzzy membership functions.

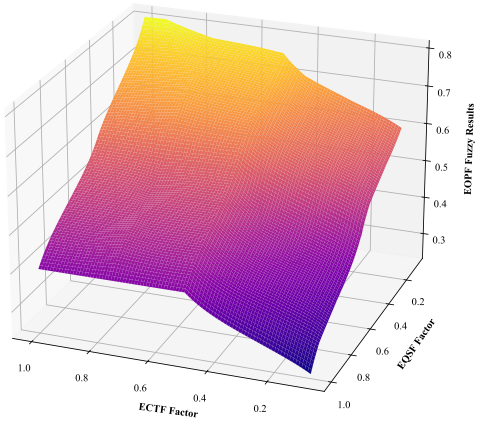


Fig. 5. Fuzzy evaluation result for various ECTF and EQSF, when LCTF is fixed at 0.5.

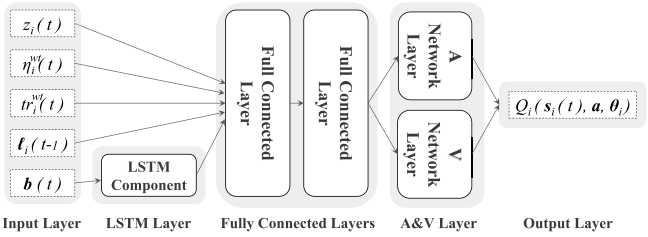


Fig. 6. Architecture of a single DQN in the Double-DQN.

cloning framework to transfer offloading expertise and support DRL agents in early learning and decision-making.

B. DRL-Based Offloading Approach

1) The Double-DQN Architecture: As shown in Fig. 6, we implement the DRL algorithm based on Rainbow-DQN, integrating Double-DQN (DDQN) [45] and Dueling-DQN [46] to improve Q-value estimation. To capture temporal patterns and system load dynamics, an LSTM [47] sub-module is embedded in the network, enabling recognition of load fluctuations at edge servers. Although experience replay is used, the LSTM operates on sequences of recent consecutive states (within T_{past} steps), preserving temporal dependencies despite random sampling. Given the dynamic nature of edge environments, long-term dependencies are often unnecessary and computationally inefficient. Instead, short-term patterns offer more practical cues for user-side decision-making. Similar techniques have been adopted in prior offloading studies [26], [43]. In Dueling-DQN, the Q-value is computed by combining the action-advantage function $A_i(\boldsymbol{s}_i(t), \boldsymbol{a}; \theta_i)$ and the state-value function $V_i(\boldsymbol{s}_i(t), \boldsymbol{a}; \theta_i)$, improving stability and avoiding

Algorithm 1 Offloading Decision-Making Stage on ud_i

Input: state $\boldsymbol{s}_i(t)$ (if task is not empty)
Output: action $a_i(t)$ and experience data

```
for episode 1 to EpisodeNum do
    for time slot t ∈ 𝒯 do
        if ud_i receives a new task k_i(t) then
            select an action a_i(t) according to Eq. (17);
        end if
        Observe the next state s_i(t+1);
        Receive information about completed tasks k_i(t');
        for task k_i(t') in k_i(t') do
            store (s_i(t'), a_i(t'), c_i(t'), s_i(t'+1)) to bf_i;
        end for
    end for
end for
```

overestimation, as follows.

$$Q_i(\boldsymbol{s}_i(t), \boldsymbol{a}; \theta_i) = V_i(\boldsymbol{s}_i(t); \theta_i) + \left(A_i(\boldsymbol{s}_i(t), \boldsymbol{a}; \theta_i) - \frac{1}{|\boldsymbol{Act}|} \sum_{\boldsymbol{a}' \in \boldsymbol{Act}} A_i(\boldsymbol{s}_i(t), \boldsymbol{a}'; \theta_i) \right). \quad (15)$$

In the DRL baseline algorithm, following the approach in [26], we adopt an $\epsilon-greedy$ strategy for action selection and use Mean Squared Error (MSE) as the loss function. During training, the parameters θ_i^{E} and θ_i^{T} of the DDQN are updated. The MSE minimizes the discrepancy between the predicted Q-values Q_i^{E} from the *Evaluate_Net* and the target Q-values $\hat{\mathbf{Q}}_i^{\text{T}} = (\hat{Q}_{i,m}^{\text{T}})_{m \in \mathcal{M}}$, computed by the *Target_Net* using a batch of experiences $\mathcal{M}$ sampled from the relay buffer bf_i. This process gradually improves the learned policy, as follows.

$$L(\theta_i^{\text{E}}, \hat{\mathbf{Q}}_i^{\text{T}}) = \frac{1}{|\mathcal{M}|} \sum_{m \in \mathcal{M}} \left(Q_i^{\text{E}}(\boldsymbol{s}_i(m), \boldsymbol{a}_i(m); \theta_i^{\text{E}}) - \hat{Q}_{i,m}^{\text{T}} \right)^2. \quad (16)$$

2) DRL-Based Offloading Algorithm: At the beginning of each time slot, if $ud_i \in \mathcal{D}$ receives a new task $k_i(t)$, it observes the environment and wrap its state data. Its agent then selects an offloading action according to Equation (17) and Algorithm 1. An $\boldsymbol{\epsilon}$-greedy strategy is used for action selection: with probability $\boldsymbol{\epsilon}$, a random action is chosen to encourage exploration; with probability $1-\boldsymbol{\epsilon}$, the action with the highest Q-value is selected. As in many DQN-based approaches, $\boldsymbol{\epsilon}$ is gradually diminished from 1 during training to shift from exploration to exploitation. Since the task $k_i(t)$ may span multiple time slots, the tasks observed as completed at slot $t+1$ may correspond to $k_i(t')$ where $t' \leq t$. Accordingly, the agent collects experience from these completed tasks and stores them in bf_i.

$$a_i(t) = \begin{cases} argmin_{a \in \boldsymbol{Act}} Q_i^{\text{E}}(\boldsymbol{s}_i(t), \boldsymbol{a}; \theta_i^{\text{E}}), & \text{with probability } 1-\boldsymbol{\epsilon}, \\ \text{choose an action randomly from } \boldsymbol{Act}, & \text{with probability } \boldsymbol{\epsilon}. \end{cases} \quad (17)$$

3) DQN Training Algorithm: During training, each agent samples a batch of experiences $\mathcal{M}$ from its replay buffer. For each $m \in \mathcal{M}$, the target Q-value $\hat{Q}_{i,m}^{\text{T}}$ is computed by summing

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 DDQN Training Stage for Agent ud_i

```
Employ the trained θ_i^E, θ_i^T by BC from fuzzy logic;
Initialize StartN, IntervalN, UpdateN and MaxN;
Set TrainN:= StepN:= 0
while TrainN < MaxN do
    if StepN > StartN and mod(StepN,IntervalN) = 0 then
        Sample a batch of experience data M from bf_i ;
        for each experience m ∈ M do
            Wrap (s_i(m), a_i(m), c_i(m), s_i(m + 1));
            Compute Q̂^T_{i,m} according to Eq. (18) ;
        end for
        Wrap Q̂^T_i: = (Q̂^T_{i,m})_{m∈M};
        Update θ^E_i to minimize L(θ^E_i, Q̂^T_i) by Eq. (16) ;
        TrainN:= TrainN + 1;
        if mod(TrainN, UpdateN) = 0 then
            θ^T_i:= θ^E_i;
        end if
    end if
end while
```

the immediate cost and the discounted future Q-value of the optimal next action. This action $\tilde{\boldsymbol{a}}_m$ is determined by selecting the maximum Q-value from *Evaluate_Net*$_i$, while the Q-value itself is estimated using *Target_Net*$_i$, is defined as follows:

$$\hat{Q}^{\mathrm{T}}_{i,m} = \boldsymbol{c}_i(m) + \gamma Q^{\mathrm{T}}_i\left(\boldsymbol{s}_i(m+1), \tilde{\boldsymbol{a}}_m; \theta^{\mathrm{T}}_i\right). \tag{18}$$

Training starts once a sufficient number of experiences (e.g., 500) are collected. The network updates are performed periodically (e.g., every 10 steps). Following the DDQN framework, the target network is updated by copying weights from the evaluation network after a fixed number of training iterations (e.g., 200). The procedure is detailed in Algorithm 2.

C. Behavior Cloning

Behavior cloning (BC), grounded in supervised deep learning [10], updates network parameters using Equation (16) based on expert demonstrations from the fuzzy logic module. The goal is to refine the agent's initial policy and improve early-stage decision-making. In this study, we implement DDQN and fuzzy logic as baselines, and introduce five BC strategies: Only-BC, Norm-BC, Fading-BC, Runtime-BC, and Hybrid-BC. For comparison, the DRL baseline without cloning is referred to as Non-BC. A unified workflow of behavior cloning from fuzzy logic is shown in Fig. 7, with strategy details as follows:

Only-BC: In this strategy, experiences (e.g., 100 episodes) generated by the fuzzy logic module are initially loaded into the replay buffer of each DRL agent. Behavior cloning is performed independently using these samples, with no subsequent reinforcement learning. Instead, the agent directly uses the cloned network parameters for task offloading. The goal is to demonstrate that an agent initialized solely through behavior cloning and early data can still exhibit basic decision-making capabilities.

Norm-BC: In this strategy, experiences generated by the fuzzy logic module are first injected into the replay buffer to support standalone behavior cloning. After cloning, the buffer is cleared, and each agent begins reinforcement learning from scratch, collecting new experiences through interaction. This design ensures learning is driven solely by the cloned policy, avoiding residual data interference and enabling a clean start.

Fading-BC: Similar to Norm-BC, this strategy begins by injecting fuzzy logic-generated experiences into the replay buffer for behavior cloning. Unlike Norm-BC, the buffer is not cleared after cloning; instead, as reinforcement learning progresses, these expert experiences are gradually replaced by newly collected samples. This fading mechanism ensures a smooth transition from imitation to autonomous learning, promoting training stability and continuity.

Runtime-BC: This strategy interleaves reinforcement learning and fuzzy logic, allowing agents to alternate in offloading tasks and to jointly accumulate experience. In order to balance the influence of expert demonstrations, the proposed strategy adopts a training schedule with twice the number of episodes compared to Norm-BC methods. Its core principle is real-time imitation, enabling the agent to observe and imitate expert actions concurrently during training.

Hybrid-BC: Building upon the strengths of both approaches, this strategy combines Fading-BC and Runtime-BC for improved task offloading performance. Initially, the agent fully clones fuzzy logic experiences to guide behavior. In the next phase, cloning intensity is halved, and fuzzy logic alternates with reinforcement learning on an episode basis. For instance, with 100 episodes of expert data, the first 50 are used for pure cloning, followed by 100 interleaved episodes (one fuzzy, one learning). This design balances expert guidance and autonomous learning, ensuring a smooth policy transition.

The five behavior cloning strategies offer varying degrees of imitation to support DRL-based task offloading. Only-BC performs full imitation without further learning. Norm-BC clones expert behavior before switching to independent learning. Fading-BC gradually shifts from imitation to autonomy. Runtime-BC alternates between imitation and learning during execution. Hybrid-BC blends full and partial imitation to balance guidance and exploration. Together, these strategies address diverse imitation needs in edge offloading.

Fig. 7 shows the three stages of using behavior cloning to enhance the initial capability of DRL. At the first stage, the fuzzy logic module is employed to offload tasks and accumulate experiences before deep reinforcement learning. At the second stage, the parameters of DDQN are updated using fuzzy logic-generated data through behavior cloning. After a specific number of behavior cloning episodes, the initial capabilities of DRL agents are rapidly enhanced. Followed by the third stage, during which reinforcement learning is applied to refine the policy.

Admittedly, as this is the first attempt to explore the use of fuzzy logic and behavior cloning to improve the early stage performance of reinforcement learning agents in edge task offloading scenario, there remains a possibility of improvement

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

TABLE III
THE MAIN CONFIGURATIONS IN BEHAVIOR CLONING STRATEGIES

BC Strategy	Fuzzy Work	BC Participator	BC Way	Clear Buffer	Continue to DRL
Only BC	Yes	Only Fuzzy	Norm	Yes	No
Norm BC	Yes	Only Fuzzy	Norm	Yes	Yes
Fading BC	Yes	Only Fuzzy	Norm	No	Yes
Runtime BC	Yes	Fuzzy & DRL	Alternately	No	Yes
Hybrid BC	Yes	Fuzzy & DRL	Norm, Alternately	No	Yes

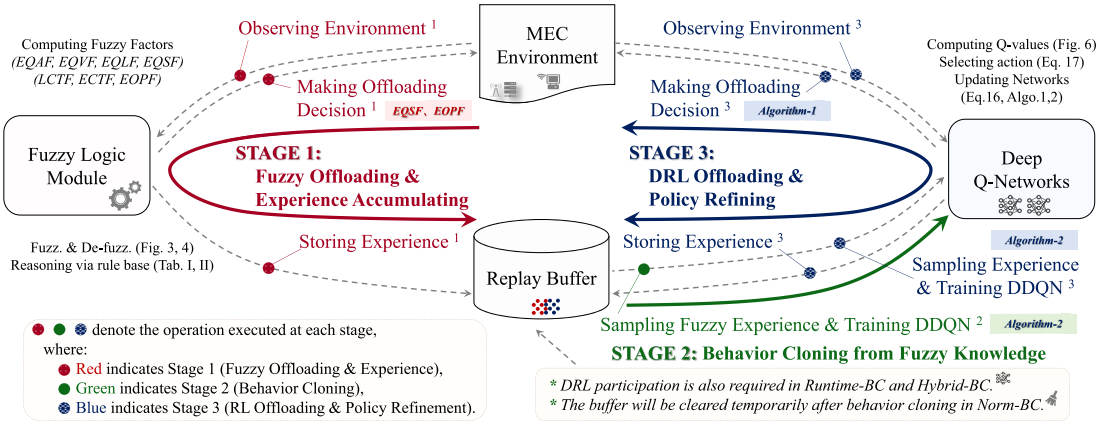


Fig. 7. Workflow of behavior cloning to enhance initial parameters and early capabilities for DRL agents.

in the design of strategies and procedures. Due to space limitations, we leave these aspects for future investigation.

D. Algorithm Analysis

Given that fuzzy logic employs simple IF-THEN rules, basic membership functions, and COG-based defuzzification, its computational overhead is negligible compared to DQN. Thus, we focus on analyzing the computational complexity of the DQN architecture, which consists of an LSTM module, fully connected (FC) layers, and a Dueling layer. For the LSTM, with time step length T_{past}, input dimension equal to the number of edge servers, and hidden state dimension H, its per-epoch complexity over a mini-batch of size $|\mathcal{M}|$ is: $\mathcal{O}(4 \cdot |\mathcal{M}| \cdot T_{past} \cdot (SH + H^2 + H))$. As input, the FC layers receive a concatenated vector of the LSTM output and the state vector, with a total dimension of $S+N$. The FC module consists of two layers, each comprising L neurons. The per-epoch complexity is: $\mathcal{O}(|\mathcal{M}| \cdot ((S+N) \cdot L + L^2))$. The Dueling layer computes state value and action advantage for A actions, yielding complexity: $\mathcal{O}(|\mathcal{M}| \cdot (L + L \cdot A))$.

Given $H = 20$, $S = 8$, $L = 128$, and $A = 6$, the dominant terms are $\mathcal{O}(T_{past}H^2)$ for the LSTM and $\mathcal{O}(L^2)$ for the FC and Dueling layers. Letting E be the number of episodes and K the steps per episode, the overall training complexity is: $\mathcal{O}(E \cdot K \cdot |\mathcal{M}| \cdot (T_{past}H^2 + L^2))$.

V. EXPERIMENTS AND RESULTS

This section first determines the EOPF threshold for the fuzzy module's offloading decisions (in Section IV, above Table II), then evaluates our baseline methods and behavior cloning strategies. We compare their performance across different user device counts and task arrival rates, analyzing variations between cloning approaches. Using episode-averaged metrics: the Average values of Task Completion Rate (ATCR) and the Average values of Task Processing Time cost (ATPT) of all user devices over the episodes during the experiment run.

A. Main Parameters

The default parameters for simulation experiment are listed in Table IV. The choice of environment settings and hyper-parameters is not arbitrary. Hyperparameters for the fuzzy logic module were selected empirically, as detailed later in this section. MEC environment and DRL configurations follow prior studies, particularly [26], [43]. For instance, the study [43] reports that increasing the batch size from 8 to 32 yields minimal performance gains, while smaller sizes shorten training time per round. Accordingly, we adopt 16 as the default batch size.

Unlike Atari games, where frames vary drastically, our environment is driven by the evolving status of edge servers. Although temporal variability exists, changes are more gradual, and feedback is obtained with relatively low delay. To capture short-term temporal patterns, we retain around 10 transitions per LSTM update. Consequently, a large replay buffer is unnecessary; in practice, a size of 500 suffices.

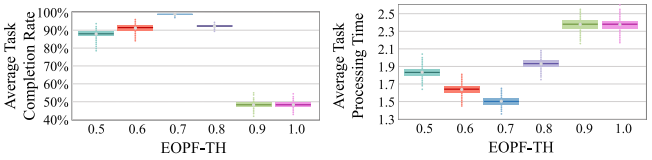
B. EOPF Threshold for Task Offloading

Fuzzy logic module uses *EOPF* for offloading decisions, where higher values favor edge processing. We test *EOPF-TH* thresholds from 0.5 to 1.0 (steps of 0.1) over 1000 episodes. Fig. 8 presents ATCR and ATPT results. The box plots reveal *EOPF-TH* = 0.7 achieves optimal balance: (a) the highest

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

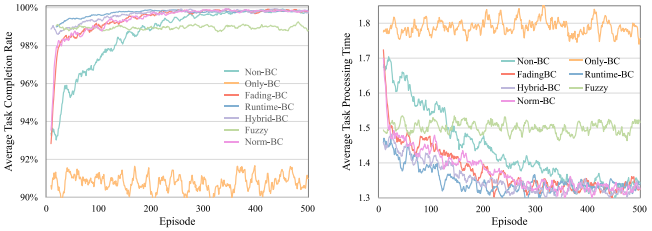
TABLE IV
DEFAULT PARAMETER SETTINGS

Parameter	Value	Parameter	Value
δ	0.1 second	$\rho_i(t)$	0.297 Gcyc./Mbits
$sz_i(t)$	2 – 5 Mbits	$r_{i,j}$	14 Mbps
$\tau_i(t)$	10 slots	f_i^{d}	2.50 GHz
Num. of UD	50, 100	f_j^{s}	41.8 GHz
Num. of ES	5	Task arrival rate	0.3
γ	0.9	Learning rate	10^{-3}
ϵ	$1 \rightarrow 0.01$	Learning period	10 steps
ϵ increment	2.5×10^{-4}	Replay buffer size	500
EOPF-TH	0.7	Slots/episode	100
Batch size	16	BC episodes	100
Act. func.	ReLU	Optimizer	Adam



(a) (b)

Fig. 8. Performance comparison for fuzzy logic module with different EOPF thresholds in terms of (a)ATCR and (b)ATPT.



(a) (b)

Fig. 9. Performance comparison for baselines and BC strategies in terms of (a)ATCR and (b)ATPT.

ATCR with minimal fluctuation and (b) the lowest ATPT. This threshold is adopted for subsequent experiments.

C. Convergence Comparison

This experiment evaluates the effectiveness of the two baseline algorithms and the capability of each behavior cloning strategies by comparing their performance with default parameters using the ATCR and ATPT metrics with 500 episodes.

1) Observations From ATCR: As shown in Fig. 9(a), all behavior cloning strategies, except Only-BC, achieve higher initial ATCR than both Non-BC (i.e., DDQN) and Fuzzy (i.e., fuzzy logic). Due to the absence of reinforcement learning, Only-BC's performance remains limited. In contrast, Norm-BC and Fading-BC apply cloned policies early, accelerating convergence faster despite slightly lower peak performance compared to Hybrid-BC and Runtime-BC. The low ATCR of Only-BC confirms that behavior cloning enhances early-stage performance but cannot substitute reinforcement learning entirely. Moreover, all strategies except Only-BC demonstrate significantly better initial performance than Non-BC and achieve convergence over 200 episodes earlier. The fuzzy logic module, though simple, maintains stable performance with an ATCR close to 99%, surpassing DDQN and validating the effectiveness of the proposed design. Behavior cloning also improves training stability, as all strategies excluding Only-BC exhibit reduced fluctuations in both ATCR and ATPT compared to Non-BC and Fuzzy.

2) Observations From ATPT: As shown in Fig. 9(b), the trends of ATPT highlights that Only-BC lacks subsequent reinforcement learning, exhibiting the largest ATPT, possibly leading to more task drops. In contrast, Fuzzy holds smaller ATPT than Non-BC and transfers its advantages to DRL agents via behavior cloning. All BC strategies, still except Only-BC, achieve significantly lower ATPT than the DRL baseline, converging faster to optimal performance. The curves in Fig. 9 also indicate that behavior cloning slightly improves the stability of the reinforcement learning, not only in early stages but also in the later learning process.

3) Observations From Computational Cost: We focus on comparing computational overhead in the early training stage. As fuzzy logic is lightweight, its cost is negligible relative to neural network computations. Assuming 100 time slots per episode, a task arrival rate of 0.3, and training every 10 slots, we estimate overhead using neural network forward passes and training times (including backpropagation). As shown in Fig. 9(a), Taking Fading-BC as an example, it first runs 100 episodes guided by fuzzy logic at minimal cost, then requires 25 additional training episodes to reach 98% ATCR. This results in approximately $25 \times 100 \times 0.3 = 750$ forward passes and $25 \times 10 = 250$ training times, totaling $1,000$ DQN operations. When scaled to 50 agents, the total number of DQN computations required is approximately $1,000 \times 50 = 50,000$, which remains substantially lower than the computational demand of the Non-BC baseline, as estimated below.

By comparison, the Non-BC baseline needs at least 135 episodes to reach the same ATCR level, with reduced learning stability. This leads to about $135 \times 100 \times 0.3 = 4,050$ forward passes and $135 \times 10 = 1,350$ training times. Each agent performs approximately $5,400$ DQN computations. For a system with 50 agents, the total number of DQN computations reaches approximately $270,000$ (i.e., $5,400 \times 50$). Hence, Fading-BC introduces no extra computational burden and may even reduce early-stage training cost for reaching the same performance compared to the DRL baseline. These advantages are especially beneficial in multi-agent or federated learning scenarios. Similar conclusions hold in the early stage for all strategies except Only-BC.

D. Episodes for Behavior Cloning

This study explores behavior cloning to enhance the early-stage performance of DRL agents. A key challenge is determining the optimal number of BC episodes: too few lead to insufficient knowledge transfer, while too many may waste resources and potentially hinder the reinforcement learning process. The goal is to equip DRL agents with initial offloading capabilities using a limited set of experiences generated by fuzzy logic, thereby minimizing reliance on fuzzy logic during subsequent learning phases. In the experiment, BC episodes ranged from 20 to 100 (step 20), and the first 50 DRL episodes

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

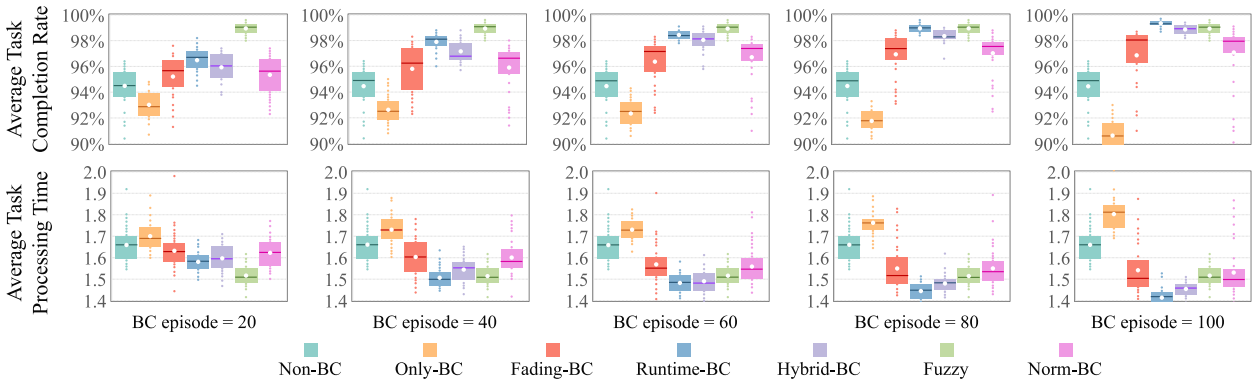


Fig. 10. Performance comparison for baselines and BC strategies with different BC episodes in terms of ATCR and ATPT.

(after BC) were evaluated using ATCR and ATPT. Multiple BC strategies were compared with baseline methods, as shown in Fig. 10 using uniformly scaled box plots. scaled box plots.

1) Observations From ATCR: In the first row of Fig. 10, the results show that the number of BC episodes leads to more significant improvement over the DRL baseline (Non-BC). For instance, with 100 BC episodes, the ATCR rises from 59%(Non-BC) to 99% (Runtime-BC). However, Only-BC, which relies solely on cloned experiences without further learning, performs the worst. Among the BC strategies, Runtime-BC and Fadeout-BC achieve the best performance, slightly outperforming others due to a few additional BC episodes. Notably, Runtime-BC is the only strategy that surpasses fuzzy logic in ATCR, achieving this after alternating 100 BC and 100 DRL episodes.

2) Observations From ATPT: As shown in the second row of Fig. 10, the ATPT trend aligns with that of ATCR. An increasing number of BC episodes leads to reduced ATPT for all BC strategies (except Only-BC) compared to the DRL baseline. For instance, with 100 BC episodes, Runtime-BC achieves an ATPT reduction of about 0.2 across all user devices, enhancing QoE in dense edge computing scenarios. Runtime-BC and Hybrid-BC begin to outperform Fuzzy after 40 episodes, with Runtime-BC achieving even lower ATPT than Fuzzy at 100 episodes. Even without dynamic strategies like Runtime-BC and Hybrid-BC, Fading-BC and Norm-BC still yield lower ATPT than the DDQN baseline, indicating faster task completion. These results demonstrate that as few as 40 to 100 BC episodes can significantly improve the early performance of DRL in both ATCR and ATPT. If the number of training epochs is increased, fewer BC episodes may be required. However, this aspect will be explored in future work.

E. Performance Comparison Across Device Quantities

In this experiment, the number of devices is varied from 50 to 150 in steps of 20. For each setting, ATCR and ATPT data were collected during the first 50 DRL episodes(after BC). Box plots in Fig. 11 illustrate both value distributions and fluctuations, using consistent axes and scales. The top row shows ATCR, and the bottom row presents ATPT.

1) Observations From ATCR: In the first row of Fig. 11, ATCR generally declines across all methods as the number of devices increases, reflecting rising system complexity. The fuzzy module consistently ranks among the top performers, validating its design effectiveness. Most BC strategies outperform the DRL baseline (Non-BC). When devices count exceeds 110, Only-BC's ATCR matches that of Non-BC, indicating behavior cloning can equip DDQN with initial capabilities even without reinforcement learning. This advantage becomes more evident with increasing device numbers and is also observed in other BC strategies. Runtime-BC consistently leads, closely followed by Hybrid-BC, primarily, due to its cloning mechanism, with some benefit from DDQN experiences. Fading-BC and Only-BC perform similarly but lag behind. Notably, Runtime-BC and Hybrid-BC retain their replay buffer after cloning, enabling continued use of fuzzy logic-based experience.

2) Observations From ATPT: In the second row of Fig. 11, ATPT increases for all methods as the number of devices grows. DDQN still falls short of matching the initial capabilities of other methods. Overall, the experiment highlights the effectiveness of behavior cloning in scenarios with more user devices, as behavior cloning strategies consistently outperform DDQN, delivering more stable and efficient offloading policies.

F. Performance Comparison Across Task Arrival Rate

While 50 devices and 30% task arrival rate are commonly used, we found this configuration is not challenging. To test algorithm performance in more complex environments, in this experiment, we set the number of devices as 100, keeping other parameters unchanged. The task arrival rate was varied from 40% to 70%(to prevent system overload) with a 10% increment. ATCR and ATPT in the first 50 episodes (after BC) are illustrated in Fig. 12.

1) Observations From ATCR: As shown in Fig. 12(left), ATCR decreases across all methods as the task arrival rate increases. Nevertheless, behavior cloning strategies consistently outperform the DRL baseline (Non-BC). When the rate is below 50%, Runtime-BC and Hybrid-BC achieve the best performance due to more frequent behavior cloning, followed by Fading-BC and Only-BC. Beyond 50%, however, Runtime-BC and Hybrid-BC underperform, likely due to the adverse effect of the fuzzy logic module under high load. Although

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

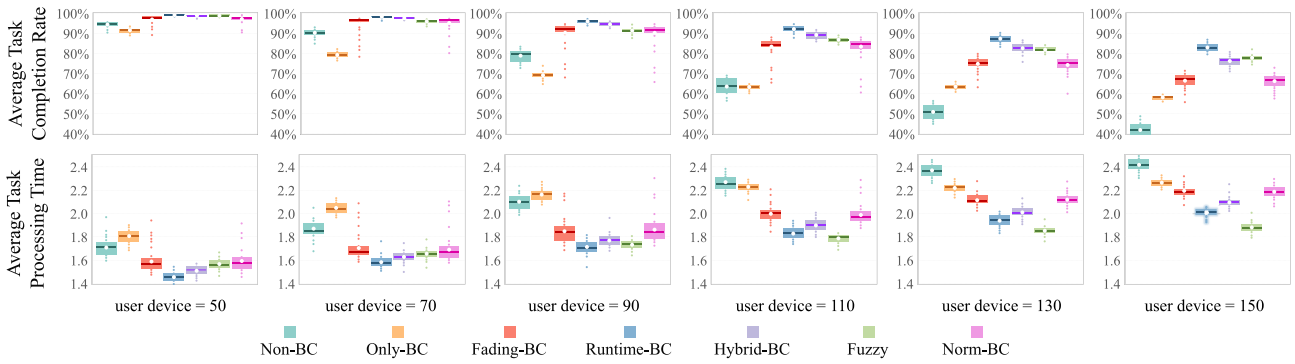


Fig. 11. Performance comparison for methods with different number of user devices in terms of ATCR and ATPT.

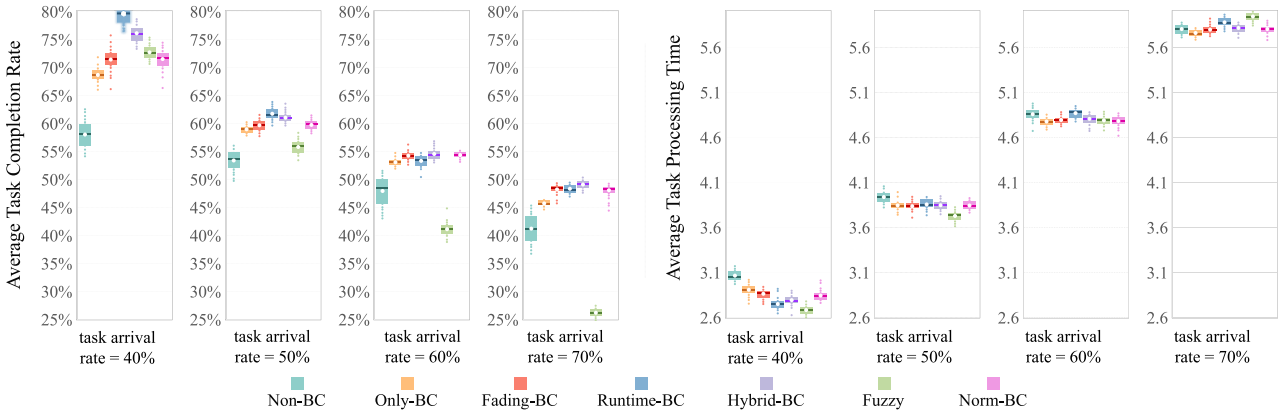


Fig. 12. Performance comparison for baselines and BC strategies with different task arrival rate in terms of ATCR and ATPT.

Only-BC lacks reinforcement learning and performs worst among BC strategies, it still initially outperforms Non-BC baseline. Fuzzy logic shows reduced robustness at higher arrival rates, partly due to its strict timeout tolerance and the fixed EOPF-TH threshold, which warrants further study for adaptive tuning. Despite these limitations, the fuzzy module does not hinder behavior cloning's effectiveness, with ATCR improvements ranging from 20% to 8% as the rate increases from 40% to 70%.

2) Observations From ATPT: As shown in Fig. 12(right), ATPT increases across all methods as the task arrival rate rises. While fuzzy logic shows some vulnerability under very high rates, it slightly outperforms Non-BC in typical cases. For rates up to 50%, most of BC strategies achieve lower ATPT than Non-BC. However, beyond this point, Runtime-BC and Hybrid-BC no longer provide the best ATPT performance. In contrast, Only-BC and Fading-BC continue to reflect the benefits of behavior cloning. Despite the fuzzy module's limitations under heavy load, behavior cloning consistently enhances initial agent performance, achieving higher task completion rates and lower processing times than both the DDQN and fuzzy module. This highlights its potential to improve system efficiency in complex scenarios.

G. Performance Comparison Across Task Size

This experiment compares the early stage performance of baseline methods and behavior cloning strategies. The average task size ranges from 2 Mbits to 10 Mbits, with a 2 Mbits increment. The performance during the first 50 episodes (after BC) in DRL are compared by ATCR and ATPT.

1) Observations From ATCR: Fig. 13(left) shows that as the average task size increases, ATCR generally declines across all methods due to rising system complexity. At 2 Mbits, tasks are sparse, and nearly all can be completed. However, when the size reaches 10 Mbits, queues become congested, many tasks timeout, and ATCR drops sharply to below 25%. Across all task sizes, DRL agents with initialized via behavior cloning consistently outperform naive DRL in ATCR. For example, at 6 Mbits, Non-BC achieves around 65% ATCR, while Norm-BC reaches approximately 75%, with Runtime-BC and Hybrid-BC performing even better. This trend holds consistently under non-extreme conditions.

2) Observations From ATPT: Fig. 13(right) shows that as the average task size increases, the ATPT also rises, growing from 0.7 at 2Mbits to 2.6 at 10Mbits. Overall, under moderate average task sizes (2 6Mbits), DRL with behavior cloning exhibits lower ATPT compared to naive DRL (Non-BC), and fuzzy logic methods also perform relatively well. For example, at the average task size of 4Mbits, the ATPT of Norm-BC and Fuzzy is about 0.1 lower than that of Non-BC (1.5 vs. 1.6). This indicates that, under general task sizes, DRL agents using behavior cloning achieve relatively better initial ATPT.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

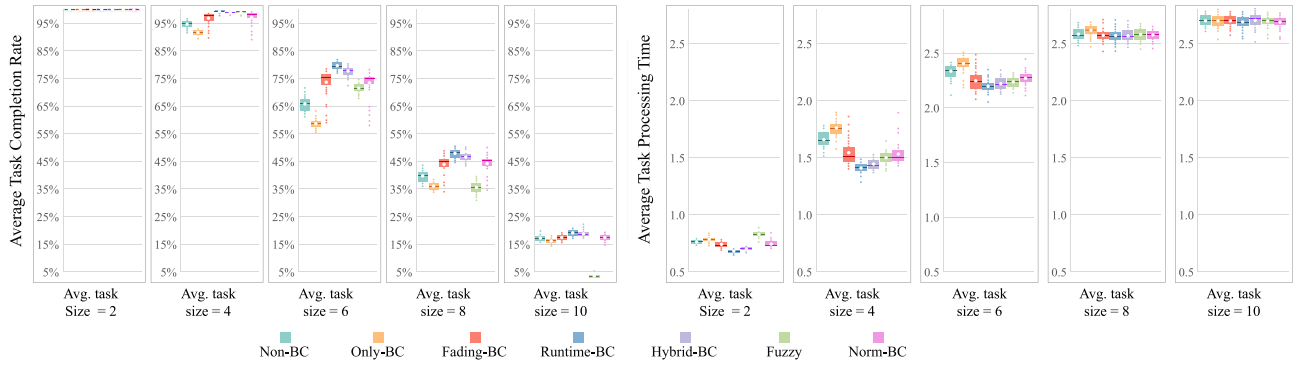


Fig. 13. Performance comparison for baselines and BC strategies with different Avg. task size in terms of ATCR and ATPT.

VI. Conclusion

This paper addresses the cold-start problem in edge task offloading, which arises from the random initialization of DRL algorithms. To mitigate this issue and enhance the initial and early performance of the DRL algorithm, we leverage the advantages of fuzzy logic, which emulates human-like thinking patterns, and apply behavior cloning to transfer these advantages to the initial DRL agents. This provides DRL agents with more behavior-driven initial parameters. Through simulation experiments, we found that, except for the only-BC strategy which lacks of reinforcement learning after BC, most of the behavior cloning strategies designed in this paper improve the early performance of the DRL algorithm, allowing DRL agents to quickly improve task completion rates and optimize task completion times in distributed computing task offloading scenarios. In addition, these strategies slightly improve the stability of the learning process. Furthermore, fuzzy logic and DDQN meet the objectives of the baseline methods, demonstrating solid task offload performance even when used independently.

Despite the promising results, this work still has certain limitations and leaves room for further exploration. Our experiments suggest that the fuzzy logic module can be further optimized, particularly in terms of adaptability. Future efforts will focus on enhancing this module and extending the framework to support a wider range of deep reinforcement learning algorithms for edge task offloading. Additionally, we plan to refine the behavior cloning process and explore its integration with fuzzy logic to efficiently generate high-quality initial models, especially in federated learning environments. These directions are expected to improve the early-stage performance and scalability of DRL-based offloading strategies.

References

[1] K. W. Zada, E. Ahmed, S. Hakak, I. Yaqoob, and A. Ahmed, "Edge computing: A survey," *Future Gener. Comput. Syst.*, vol. 97, pp. 219–235, Aug. 2019.

[2] L. A. Haibeh, M. C. E. Yagoub, and A. Jarray, "A survey on mobile edge computing infrastructure: Design, resource management, and optimization approaches," *IEEE Access*, vol. 10, pp. 27591–27610, 2022.

[3] L. Huang, X. Feng, L. Zhang, L. Qian, and Y. Wu, "Multi-server multi-user multi-task computation offloading for mobile edge computing networks," *Sensors*, vol. 19, no. 6, p. 1446, 2019.

[4] C. Jiang, X. Cheng, H. Gao, X. Zhou, and J. Wan, "Toward computation offloading in edge computing: A survey," *IEEE Access*, vol. 7, pp. 131543–131558, 2019.

[5] T. X. Tran and D. Pompili, "Joint task offloading and resource allocation for multi-server mobile-edge computing networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 1, pp. 856–868, Jan. 2019.

[6] S. Long, Y. Zhang, Q. Deng, T. Pei, J. Ouyang, and Z. Xia, "An efficient task offloading approach based on multi-objective evolutionary algorithm in cloud-edge collaborative environment," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 2, pp. 645–657, Mar./Apr. 2023.

[7] R. Men, X. Fan, A. Shan, and G. Yuan, "Fuzzy logic based binary computation offloading scheme in V2X communication networks," *IEEE Access*, vol. 12, pp. 45507–45518, 2024.

[8] A. Choudhury, M. Ghose, A. Islam, and Yogita, "Machine learning-based computation offloading in multi-access edge computing: A survey," *J. Syst. Archit.*, vol. 148, Mar. 2024, Art. no. 103090.

[9] H. Xiao, Z. Hu, X. Zhang, A. Xu, M. Zheng, and K. Li, "Federated deep reinforcement learning for task offloading in MEC-enabled heterogeneous networks," *IEEE Internet Things J.*, vol. 12, no. 8, pp. 10238–10252, Apr. 2025.

[10] B. Zheng, S. Verma, J. Zhou, I. W. Tsang, and F. Chen, "Imitation learning: Progress, taxonomies and challenges," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 35, no. 5, pp. 6322–6337, May 2024.

[11] A. O. Ly and M. Akhloufi, "Learning to drive by imitation: An overview of deep behavior cloning methods," *IEEE Trans. Intell. Veh.*, vol. 6, no. 2, pp. 195–209, Jun. 2021.

[12] P. Florence et al., "Implicit behavioral cloning," in *Proc. 5th Conf. Robot Learn.*, 2022, pp. 158–168.

[13] J. Zhang, Y. Lin, Z. Du, and W. Bao, "Fuzzy logic-based deep reinforcement learning pre-training: A case study in MEC task offloading," in *Proc. IEEE Cyber Sci. Technol. Congr. (CyberSciTech)*, 2024, pp. 23–30.

[14] A. Bozorgchenani, D. Tarchi, and G. E. Corazza, "Centralized and distributed architectures for energy and delay efficient fog network-based edge computing services," *IEEE Trans. Green Commun. Netw.*, vol. 3, no. 1, pp. 250–263, Mar. 2019.

[15] F. Yu et al., "Federated deep reinforcement learning-enabled task offloading in cloud-edge-terminal collaborative networks," in *Proc. IEEE 99th Veh. Technol. Conf.*, 2024, pp. 1–5.

[16] C. Ren, G. Zhang, X. Gu, and Y. Li, "Computing offloading in vehicular edge computing networks: Full or partial offloading?" in *Proc. IEEE 6th Inf. Technol. Mechatronics Eng. Conf.*, 2022, pp. 693–698.

[17] Y. Deng, Z. Chen, X. Chen, and Y. Fang, "Task offloading in multi-hop relay-aided multi-access edge computing," *IEEE Trans. Veh. Technol.*, vol. 72, no. 1, pp. 1372–1376, Jan. 2023, doi: 10.1109/TVT.2022.3204398.

[18] G. Wang, S. Zhou, and Z. Niu, "Radio resource allocation for bidirectional offloading in space-air-ground integrated vehicular network," in *J. Commun. Inf. Netw.*, vol. 4, no. 4, pp. 24–31, Dec. 2019.

[19] Z. Zabihi, A. M. E. Moghadam, and M. H. Rezvani, "Reinforcement learning methods for computation offloading: A systematic review," *ACM Comput. Surv.*, vol. 56, no. 1, pp. 1–41, 2024.

[20] S. Wang, T. Qin, T. Chen, W. Guo, Y. Hu, and H. Sun, "A high-reliability small-area task offloading mechanism with trust evaluation and fuzzy logic in power IoTs," *IEEE Trans. Mobile Comput.*, vol. 24, no. 4, pp. 2935–2948, Apr. 2025.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.

[21] S. Zhang, N. Yi, and Y. Ma, "A survey of computation offloading with task types," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 8, pp. 8313–8333, Aug. 2024.
[22] S. Wu, N. Chen, G. Wen, L. Xu, P. Zhang, and H. Zhu, "Virtual network embedding for task offloading in IIoT: A DRL-assisted federated learning scheme," *IEEE Trans. Ind. Informat.*, vol. 20, no. 4, pp. 6814–6824, Apr. 2024.
[23] Y. Dai, J. Zhao, J. Zhang, Y. Zhang, and T. Jiang, "Federated deep reinforcement learning for task offloading in digital twin edge networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 3, pp. 2849–2863, May/Jun. 2024.
[24] S. Pang, T. Wang, H. Gui, X. He, and L. Hou, "An intelligent task offloading method based on multi-agent deep reinforcement learning in ultra-dense heterogeneous network with mobile edge computing," *Comput. Netw.*, vol. 250, Aug. 2024, Art. no. 110555.
[25] Y. Liang, H. Tang, H. Wu, Y. Wang, and P. Jiao, "Lyapunov-guided offloading optimization based on soft actor-critic for ISAC-aided Internet of Vehicles," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 14708–14721, Dec. 2024.
[26] I. Rahmati, H. Shah-Mansouri, and A. Movaghar, "QECO: A QoE-oriented computation offloading algorithm based on deep reinforcement learning for mobile edge computing," 2024, *arXiv:2311.02525*.
[27] P. Peng et al., "A survey on computation offloading in edge systems: From the perspective of deep reinforcement learning approaches," *Comput. Sci. Rev.*, vol. 53, Aug. 2024, Art. no. 100656.
[28] D. H. Abdulazeez and S. K. Askar, "Offloading mechanisms based on reinforcement learning and deep learning algorithms in the fog computing environment," *IEEE Access*, vol. 11, pp. 12555–12586, 2023.
[29] A. Mahapatra, S. K. Majhi, K. Mishra, R. Pradhan, D. C. Rao, and S. K. Panda, "An energy-aware task offloading and load balancing for latency-sensitive IoT applications in the fog-cloud continuum," *IEEE Access*, vol. 12, pp. 14334–14349, 2024.
[30] D. H. Abdulazeez and S. K. Askar, "A novel offloading mechanism leveraging fuzzy logic and deep reinforcement learning to improve IoT application performance in a three-layer architecture within the fog-cloud environment," *IEEE Access*, vol. 12, pp. 39936–39952, 2024.
[31] V. Nguyen, T. T. Khanh, T. Z. Oo, N. H. Tran, E.-N. Huh, and C. S. Hong, "Latency minimization in a fuzzy-based mobile edge orchestrator for IoT applications," *IEEE Commun. Lett.*, vol. 25, no. 1, pp. 84–88, Jan. 2021.
[32] B. Lu, S. Lin, and J. Shi, "TDMA-NOMA based computation offloading for cognitive capacity harvesting networks with transmission order optimization," *IEEE Trans. Commun.*, vol. 70, no. 9, pp. 6355–6369, Sep. 2022.
[33] K. Rai, S. Vemireddy, and R. R. Rout, "Fuzzy logic based task scheduling algorithm in vehicular fog computing framework," in *Proc. IEEE 18th India Council Int. Conf.*, 2021, pp. 1–6.
[34] S. Khan, Z. Jiangbin, and H. Ali, "Soft computing approaches for dynamic multi-objective evaluation of computational offloading: A literature review," *Clust. Comput.*, vol. 27, no. 9, pp. 12459–12481, 2024.
[35] K. Khalida, "A review of computation offloading for mobile cloud computing based on fuzzy set theory," *Int. J. Eng. Trends Technol*, pp. 56–63, Jan. 2020, doi: 10.14445/22315381/CATI3P209.
[36] S. R. Behera, N. Panigrahi, S. K. Bhoi, M. Bilal, K. S. Sahoo, and D. Kwak, "A distributed fuzzy optimal decision making strategy for task offloading in edge computing environment," *IEEE Access*, vol. 11, pp. 33189–33204, 2023.
[37] D. B. Son, V. T. An, T. T. Hai, B. M. Nguyen, N. P. Le, and H. T. T. Binh, "Fuzzy deep Q-learning task offloading in delay constrained vehicular fog computing," in *Proc. Int. Joint Conf. Neural Netw.*, 2021, pp. 1–8.
[38] X. Wang, Z. Ning, S. Guo, and L. Wang, "Imitation learning enabled task scheduling for online vehicular edge computing," *IEEE Trans. Mobile Comput.*, vol. 21, no. 2, pp. 598–611, Feb. 2022.
[39] X. Xu et al., "Game theory for distributed IoV task offloading with fuzzy neural network in edge computing," *IEEE Trans. Fuzzy Syst.*, vol. 30, no. 11, pp. 4593–4604, Nov. 2022.
[40] X.-F. Sui, J.-S. Wang, S.-H. Zhang, S.-W. Zhang, and Y.-H. Zhang, "Multi-strategy fusion mayfly algorithm on task offloading and scheduling for IoT-based fog computing multi-tasks learning," *Artif. Intell. Rev.*, vol. 58, no. 5, p. 144, 2025.
[41] N. Zhao, Z. Ye, Y. Pei, Y.-C. Liang, and D. Niyato, "Multi-agent deep reinforcement learning for task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 21, no. 9, pp. 6949–6960, Sep. 2022.
[42] Y. Fan, J. Ge, S. Zhang, J. Wu, and B. Luo, "Decentralized scheduling for concurrent tasks in mobile edge computing via deep reinforcement learning," *IEEE Trans. Mobile Comput.*, vol. 23, no. 4, pp. 2765–2779, Apr. 2024.
[43] M. Tang and V. W. S. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," *IEEE Trans. Mobile Comput.*, vol. 21, no. 6, pp. 1985–1997, Jun. 2022.
[44] W. Van Leekwijck and E. E. Kerre, "Defuzzification: Criteria and classification," *Fuzzy Sets Syst.*, vol. 108, no. 2, pp. 159–178, 1999.
[45] H. V. Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, 2016, pp. 2094–2100.
[46] Z. Wang, T. Schaul, M. Hessel, H. Hasselt, M. Lanctot, and N. Freitas, "Dueling network architectures for deep reinforcement learning," in *Proc. 33rd Int. Conf. Mach. Learn.*, 2016, pp. 1995–2003.
[47] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.



Jiujie Zhang (Graduate Student Member, IEEE) received the B.S. degree in communication engineering and the M.S. degree in computer science and technology from Inner Mongolia Normal University, Hohhot, China, in 2013 and 2016, respectively. He is currently pursuing the Doctoral degree with the Department of Computer and Network Engineering of The University of Electro-Communications, Tokyo, Japan. Since 2016, He has been working as a Lecturer with the Faculty of Information Science and Technology, Baotou Teachers' College, Baotou, China. His research interests include Edge Computing and the Internet of Vehicles.



Yangfei Lin (Member, IEEE) received the B.E. degree from the Nanjing University of Posts and Telecommunications, Nanjing, China, in 2013, the M.E. degree from Fuzhou University, Fuzhou, China, in 2016, and the Ph.D. degree from Tsukuba University, Tsukuba, Japan, in 2022. She is currently a Postdoctoral Researcher with The University of Electro-Communications, Japan. Her research interests include cloud computing security, privacy preserving, and semantic communications.



Zhaoyang Du (Member, IEEE) received the Ph.D. degree from The University of Electro-Communications, Tokyo, Japan, in 2021, where he is currently a Postdoctoral Researcher. His current research interests include delay-tolerant networks, vehicular ad hoc networks, and IoT.



Lei Zhong (Member, IEEE) is a Principal Researcher with the InfoTech Division, Toyota Motor Corporation, where he leads research and standardization initiatives focused on intelligent connected vehicles. He represents Toyota as a chief delegate to several major standardization bodies, including 3GPP, IEEE, and the Wi-Fi Alliance. He serves as the Chair of both the Liaison Committee and the Proof of Concept Committee within the Automotive Edge Computing Consortium. In these roles, he fosters collaboration among industry stakeholders and standardization organizations to promote open innovation and accelerate the advancement of the connected vehicle ecosystem. His research interests span wireless networking, edge computing, machine learning, connected vehicles, the Internet of Things, and big data analytics. He has authored more than 80 technical publications and holds over 40 patent applications in these fields. He also contributes to academia as the Vice-Chair of the IEEE TCGCC Special Interest Group on Green Internet of Vehicles, and as an Editor for *Vehicular Communications* (Elsevier), IEEE INTERNET COMPUTING, and the IEEE OPEN JOURNAL OF THE COMPUTER SOCIETY.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.



Wugedele Bao received the Ph.D. degree from the Minzu University of China in 2018. He is currently a Professor with the School of Computer Science and Information Engineering and a member of the Trustworthy Big Data and AI Academician Workstation, Hohhot Minzu College, Hohhot, China. His current research interests include delay-tolerant networks, vehicular ad hoc networks, and IoT.



Ha Si received the B.S. degree in computer science and technology from the Heilongjiang University, Harbin, China, in 1997, and the M.S. degree in educational technology and the Ph.D. degree in computer science and technology from Inner Mongolia Normal University, Hohhot, China, in 2007 and 2013, respectively. He is currently a Professor with the School of Computer Science and Information Engineering, Inner Mongolia Normal University, where he is a member of the Trustworthy Big Data and AI Academician Workstation. His research interests include Artificial Intelligence and NLP.



Soufiene Djahel (Senior Member,IEEE) is a leading academic in connected and autonomous systems, currently serving at Coventry University's Centre for Future Transport and Cities. With the Ph.D. degree in computer science from the University of Lille and degrees from Algerian universities, he has held academic and research positions in the U.K. and Ireland. His work focuses on advancing communication, planning, optimization, and cybersecurity in emerging mobile wireless technologies, with the goal of enabling smarter, safer, and more sustainable cities. He has led and contributed to numerous interdisciplinary research projects supported by organizations like the British council, EPSRC, Innovate UK, JSPS, and the Transport Systems Catapult. A recognized expert in his field, he frequently serves in leadership roles at IEEE-sponsored conferences, acts as guest editor for high-impact journals, and evaluates research proposals for national funding agencies in the U.K., Ireland, Switzerland, the Czech Republic, and The Netherlands. In recognition of his research excellence, he was awarded the prestigious JSPS Invitational Fellowship for Research in Japan in 2021 by the Japan Society for the Promotion of Science.



Celimuge Wu (Senior Member, IEEE) received the Ph.D. degree from The University of Electro Communications, Japan. He is currently a Professor and the Director of the Meta-Networking Research Center, The University of Electro-Communications, where he is Core Member of the Advanced Wireless and Communication Research Center. His research interests include vehicular networks, edge computing, IoT, and AI for wireless networking and computing. He is a recipient of the 2021 IEEE Communications Society Outstanding Paper Award, the 2021 IEEE Internet of Things Journal Best Paper Award, the IEEE Computer Society 2020 Best Paper Award, and the IEEE Computer Society 2019 Best Paper Award Runner-Up. He serves as an Associate Editor for IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING, IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING, and IEEE TRANSACTIONS ON GREEN COMMUNICATIONS AND NETWORKING. He is the Vice Chair (Asia Pacific) of IEEE Technical Committee on Big Data. He is an IEEE Vehicular Technology Society Distinguished Lecturer.

Authorized licensed use limited to: Eotvos Lorand University Budapest. Downloaded on October 27,2026 at 12:05:48 UTC from IEEE Xplore. Restrictions apply.